PROGRAMAÇÃO FRONT END

JS

Prof. Esp. Dacio F. Machado

</a> BOA NOiTCH! <\a>

OBJETIVO DA AULA

FAZER AS COISAS FUNCIONAREM

→ manipulando elementos com HTML com JS

Estruturas de Dados

- **Objetos**: um objeto é uma coleção de pares de nome/valor ou um texto com um mapa de valores. Exemplo: **var livro = { assunto: “JavaScript”, nPaginas: 300 };**. A forma com a qual você acessa essas propriedades são as seguintes: **livro.assunto** ou **livro[“nPaginas”] .**
- **Arrays**: também conhecidos como vetores, são uma outra forma de se trabalhar valores em JavaScript. Exemplo: **var turnos = [“manhã”, “tarde”, “noite”];**, em que se pode acessar as informações da seguinte forma: **turnos[0]**, que exibirá “manhã”. E, também, há funções específicas para esse tipo, como **turnos.lenght**, que retorna o tamanho do array.

Pilares da Orientação a Objetos

O **Paradigma de Orientação a Objetos** utiliza, em sua essência, os conceitos de **classes** e **objetos**, pois, quando se programa em uma linguagem orientada a objetos, você tenta resolver um **problema do mundo real** para o computador

O Paradigma da Orientação a Objetos traz, em seu conceito, quatro pilares essenciais:

POO

ABSTRAÇÃO

Representações do mundo real, ou seja, a partir de um objeto real do nosso dia a dia você tem que imaginar como esse objeto se comportará dentro de sua aplicação.

HERANÇA

Herança é uma maneira de reutilizar o código. Nela, uma nova classe é criada, absorvendo os membros de uma classe já existente, que são aprimorados com novas ou melhores capacidades.

POLIMORFISMO

Podemos criar métodos dentro de uma classe que reagem ou funcionam de forma diferente dependendo do objeto que é executado.

ENCAPSULAMENTO

O objetivo deste pilar é restringir a visibilidade. Você sabe que tem a propriedade ou o método, mas não precisa saber como ele faz, somente o que ele faz.

```
class Carro {
 constructor(marca, modelo, caracteristicas) {
   this.Marca = marca;
   this.Modelo = modelo;
   this.Caracteristicas = caracteristicas;
 }
 exibirDetalhes() {
   alert(`Marca: ${this.Marca} - Modelo: ${this.Modelo}`);
 }
}
const carro1 = new Carro("Ford", "Ka", ["Preto", 1.0, "2
portas"]);
carro2.exibirDetalhes();
```

Objetos

Objetos JavaScript são simplesmente coleções de pares nome-valor. Como tal, eles são similares à:

Dicionários em Python

Hashes em Perl e Ruby

Hash tables em C e C++

HashMaps em Java

Objetos

Esse tipo de estrutura de dados é largamente utilizada, o que atesta sua versatilidade. Uma vez que tudo em JavaScript é um objeto (tipos básicos principais), qualquer programa JavaScript naturalmente envolve uma grande quantidade de buscas em tabelas hash, o que é algo bom, pois elas são bem rápidas!

Junto com objetos, funções são os componentes principais para o entendimento do JavaScript. A função mais básica não poderia ser mais simples:

Funções

A definição da função (também chamada de declaração de função) consiste no uso da palavra chave function, seguida por:

Nome da Função.

Lista de argumentos para a função, entre parênteses e separados por vírgulas.

Declarações JavaScript que definem a função, entre chaves { }.

```
function sqt(numero) {
return numero * numero;
};
var x = sqt(4); //x recebe o valor
16
```

Manipulando Elementos com JS

O DOM HTML (Modelo de Objeto de Documento HTML) é um modelo de objeto para documentos HTML .

Quando uma página da web é carregada, o navegador cria uma representação em forma de árvore do documento HTML.

O DOM HTML pode ser usado para acessar elementos HTML.

A forma mais comum de acessar um elemento HTML é usando o atributo `id` iddo elemento

```html
<html>
<body>
<p id="demo"></p>
<script>
const myPara =
document.getElementById("demo");
myPara.innerHTML = "Hello
World!";
</script>
</body>
</html>
```

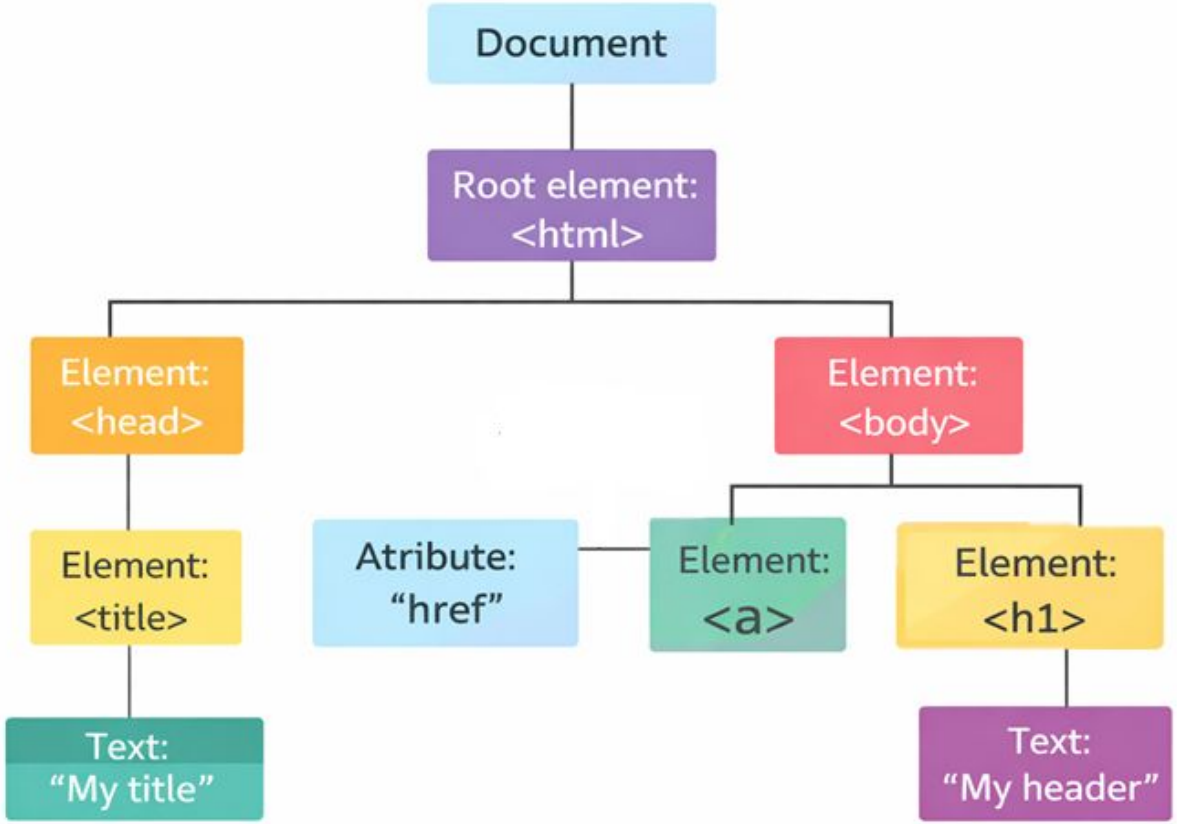
`document`é o documento HTML .

`getElementById()` é um método de documento .

`myPara = getElementById("demo")` recupera o elemento "demo" .

`innerHTML`é uma propriedade do elemento .

`myPara.innerHTML = "Hello World!"` altera a propriedade .


Document
Root element:
<html>
Element:
<head>
Element:
<body>
Element:
<title>
Atribute:
“href”
Element:
<a>
Element:
<h1>
Text:
“My title”
Text:
“My header”

Acessando o conteúdo do HTML

element.innerHTML	O conteúdo HTML de um elemento
element.textContent	O texto contido em um elemento

Atributos HTML

element.*attribute*	Alterar o valor do atributo de um elemento HTML
element.style.*property*	O estilo de um elemento HTML
element.setAttribute()	Criar ou definir um novo atributo

APLICANDO

OBRIGADO

daciofmf@gmail.com